

margin=2cm

ÉCOLE SUPÉRIEURE POLYTECHNIQUE (ESP)

DIRECTION DES ÉTUDES — NOUAKCHOTT

Filière : SID45 Big Data Processing

Real-time Banking Fraud Detection

Technical Documentation — TP5

Tech Stack

Apache Kafka • Spark Structured Streaming • Python • Jupyter •
Docker Compose

Prepared by:

24401,24406,24407,24408

Année Académique 2025–2026

Mis à jour en Mai 2026

1 Introduction

1.1 Business context

Bank X struggles to detect fraud in real time. Internet scams are increasing; reactive detection erodes customer trust and generates costly legal procedures. The bank must handle up to **1,000+ transactions per second** during peak hours while computing per-user statistics for fraud monitoring.

1.2 Project objective

This project delivers an **end-to-end streaming pipeline** designed to:

- Simulate realistic banking transactions (300k users in full specification).
- Stream JSON events through highly-available Kafka topics.
- Aggregate fraud-detection metrics with Spark Structured Streaming.
- Expose a live Jupyter dashboard for real-time monitoring and threat analysis.

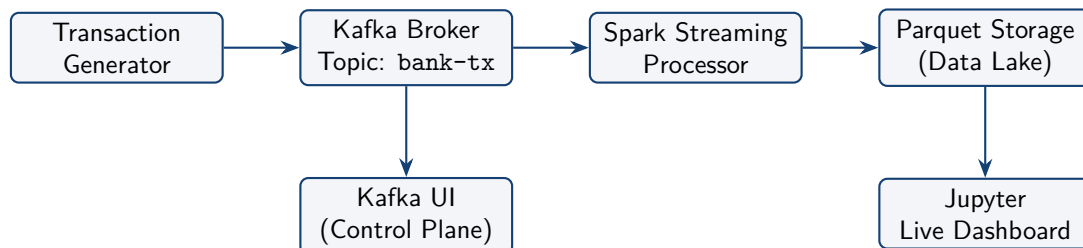
1.3 Repository layout & Deliverables

- README.md — Overview, setup requirements, and quick start guide.
- getting_started.md — Step-by-step execution walkthrough with expected traces.
- docker-compose.yml — Multi-container infrastructure definition.
- src/ — Python source codes for both the generator and Spark processor.
- notebooks/fraud_dashboard.ipynb — Real-time analytics dashboard.
- docs/architecture.md — Markdown format of this structural blueprint.

2 Architecture

2.1 High-level data flow

The following diagram illustrates the decoupled architecture from ingestion to visualization:



2.2 Docker infrastructure services

2.3 File system structure (TP5 workspace)

```
tp5/
    generator/
        transaction_generator.py
    processor/
        fraud_stream_processor.py
    dashboard_io.py
    docker-compose.yml
    getting_started.md
```

Table 1: Infrastructure components and networking layout

Service Name	Functional Role	Host Exposed Ports
transaction-generator	Python simulator → Publishes to Kafka	—
kafka	Distributed Message broker (6 partitions)	9092, 9094
kafka-ui	Cluster Management & Topic Inspection	8080
spark-master	Standalone Cluster Coordinator	7077, 8081
spark-worker-1/2/3	Distributed Scale-out Executors	8082-8084
spark-processor	Continuous Structured Streaming Job	—
jupyter	Dashboard Notebook Environment	8888

```

notebooks/
  fraud_dashboard.ipynb
data/tp5/output/          # Runtime Parquet outputs (
gitignored)
  checkpoints/tp5/        # Spark WAL state checkpoints (
gitignored)

```

3 Message Schema Contract (Bank X)

Every transaction payload published to the `bank-transactions` topic strictly adheres to the following standard JSON format:

Listing 1: Sample transaction event schema

```

1 {
2   "msg_entity": "bank_X",
3   "app_type": "mobile_app",
4   "send_entity": "bank_X",
5   "receive_entity": "bank_A",
6   "send_id": "client_12345",
7   "receive_id": "user_67890",
8   "amount": 1500.50,
9   "date": "2025-04-28T14:32:15Z",
10  "tx_type": "transfer",
11  "tx_id": "c4b370da-bfbc-49b7-bf8d-d779b5b293d2"
12 }

```

4 Transaction Generator Engine

Script path: `tp5/generator/transaction_generator.py`

4.1 Population model

The simulation features structural entities to replicate real banking traffic dynamics:

- N native clients of Bank X (`client_*`)
- M external counterparties evenly distributed between auxiliary entities Bank A and Bank B (`user_a_*`, `user_b_*`)
- **Profiles:** *Development scale:* $N = 5,000$, $M = 10,000$. *Production scale:* $100,000 + 200,000$.

4.2 Statistical distributions

To simulate realistic financial human behaviors, the generator leverages several mathematical distributions:

- **Monthly income (I):** Modelled using a discrete power law distribution $P(I) \propto 1/I^2$ bound on $[1000, 10^6]$ MRU.
- **Transaction amount (S):** Uniform distribution conditioned on income: $S \sim U[I/1000, I/100]$.
- **Frequency (f):** Number of actions computed as $f = I/S$ (transactions/month).
- **Account Balance (B):** Pre-seeded via $B \sim U[0, 3I]$.
- **Time-slice probability (p):** Probability per second calculated as $p = f/(30 \times 24 \times 3600)$, dynamically scaled by a `PEAK_MULTIPLIER` during heavy hours (09:00–12:00, 17:00–20:00 UTC).

4.3 Deployment commands

```
1 docker compose up -d transaction-generator
2 docker logs -f transaction-generator
```

5 Spark Streaming Processor

Script path: `tp5/processor/fraud_stream_processor.py`

5.1 Processing pipelines

The execution plan follows modern streaming design patterns:

1. Subscribe to Kafka topic `bank-transactions` read-stream.
2. Parse JSON dynamically; apply strict type casting and infer `event_time` from the `date` attribute.
3. Flatten financial records: explode each original item into two separate transaction perspective rows: `sent` and `received`.
4. Spin up 6 concurrent decoupled streaming micro-batch queries (Triggered at 10 seconds intervals).

5.2 Analytical Metrics Matrix

Table 2: Calculated features for fraud profiling

Aggregation Depth	Computed Metrics
Sliding Windows	<code>avg_amount</code> , <code>tx_count</code> , <code>total_amount</code> , <code>distinct_counterparties</code>
Stateful Lifetime	<code>lifetime_avg_amount</code> , <code>lifetime_tx_count</code> , <code>lifetime_total_amount</code>
Recent Traces	Raw appended user-centric events optimized for analytical querying

5.3 Sliding window horizons

5.4 I/O Target Data Lake structures

Table 3: Stateful aggregation window intervals

Window Identity	Time Range Duration	Sliding Step Dimension
3_hours	3 Hours	1 Minute
7_days	7 Days	1 Hour
3_weeks	21 Days	6 Hours
3_months	90 Days	1 Day

```
/workspace/data/tp5/output/recent_transactions/
/workspace/data/tp5/output/lifetime/
/workspace/data/tp5/output/windowed/{3_hours,7_days,3_weeks,3_months}/
/workspace/checkpoints/tp5/
```

5.5 Deployment commands

```
1 docker compose up -d spark-processor
2 docker logs -f spark-processor
```

6 Jupyter Monitoring Dashboard

Notebook path: notebooks/fraud_dashboard.ipynb

6.1 Rendered KPI components

- **Real-time Ledger:** Displays the last 20 active transactional user profiles.
- **Ingestion Velocity:** Monitor metric counts processed during the last 10 seconds.
- **Historical Context:** Multi-tab view of windowed analytical tables.
- **Anomalies flagging:** Conditional visual styling highlighting metrics that break statistical boundaries (marked in **Yellow/Red**).
- **Charts:** Live bar charts presenting comparative Sent vs Received total volumes.

6.2 Standard Operating Procedure (SOP) in Jupyter

To ensure correct initialization, notebooks must follow this setup sequence:

1. Execute **Kernel** → **Restart**.
2. Run **Cell 1** (Core packages imports & `dashboard_io` loading).
3. Run **Cell 2** (CSS UI conditional styling helpers).
4. Run **Cell 3** (`render_dashboard` engine definition).
5. Run **Cell 4** (`ipywidgets` reactive automatic refresh loop, set at 5s).

Access Link: <http://localhost:8888> — Access Token pass: **ChangeMeStrong**

7 Deployment & Operations Guide

7.1 Infrastructure bootstrap

```

1 cd spark-lab
2 docker compose up -d --build
3 ./scripts/verify_tp5.sh

```

7.2 Network endpoints

Table 4: Web Interfaces and Monitoring endpoints

Component	Target Endpoint Web Address	Operational Context
Kafka UI	http://localhost:8080	Telemetry and message inspection
Spark Master	http://localhost:8081	Cluster processing resources & job state
Jupyter Hub	http://localhost:8888	Analytical monitoring UI

7.3 Operational flow validation

Verify parts sequentially: **Kafka Broker Ready** → **Generator streaming events** → **Spark computing micro-batches** (Allow 30–60s initialization lag for Parquet metadata generation) → **Open Jupyter Dashboard**.

7.4 System cleanup and Reset procedure

CAUTION: Destructive Action

To wipe cluster state, drop corrupt offsets, and start fresh:

```

docker compose down
rm -rf data/tp5/output/* checkpoints/tp5/*
docker compose up -d --build

```

8 Core Architectural Decisions

- **Resource capping:** Default population models are scaled down for standard student development machines; env configurations allow scale out to production thresholds.
- **Generator optimizations:** Replaced slow iterative user-generation loops with high-throughput **Vectorized NumPy arrays** code blocks.
- **Sinks handling:** Window aggregates are processed using standard **append** structural rules. Lifetime metrics leverage a customized **foreachBatch** function to override Parquet storage constraints on direct **update** routines.
- **Late-Data policy:** Configured a **10-minute watermark** on event times to allow robust window recovery over lagging events.